

Fynd AI Engineer Intern Assessment 2.0

Name: Dinakar Raju Saipogu

Date: January 7, 2026

Project: AI-Powered Customer Feedback System

Note: **API Rate Limiting:** The application currently operates on the Google Gemini API's free tier, which imposes a strict quota on Requests Per Minute (RPM). During periods of rapid submission (high concurrency), the API may return a `429 Resource Exhausted` error. The system is designed to handle this gracefully: rather than crashing, it logs the error and saves the user's review to the database without the AI-generated insights, ensuring the core user experience remains uninterrupted.

1. Overall Approach

To address the assessment requirements, I adopted a modular full-stack development strategy split into two distinct phases:

- **Experimental Phase (Task 1):** I utilized a Jupyter Notebook to systematically evaluate various prompt engineering techniques. This allowed for rapid iteration and quantitative testing of the Gemini API's capabilities before implementation.
- **Deployment Phase (Task 2):** I built a lightweight, production-ready web application using Flask. The focus was on creating a robust system that integrates the bestperforming prompt strategy from Phase 1 into a user-friendly interface. The application handles real-time data persistence (SQLite) and secure cloud deployment (Render).

2. Design and Architecture Decisions

Architecture

The system follows a standard **Model-View-Controller (MVC)** pattern tailored for AI integration:

- **Frontend (View):** Built with HTML5 and Tailwind CSS for a responsive, clean user interface. It includes two distinct dashboards: a public "User Feed" for submission and a secure "Admin Panel" for analysis.

- **Backend (Controller):** Developed using **Python Flask**. I chose Flask for its minimalist nature, allowing for rapid API development and easy integration with the `googlegenerativeai` library.
- **Database (Model): SQLite** (via SQLAlchemy) was selected for data persistence. This ensures that customer reviews are saved permanently, even if the application restarts. For this scale of application, SQLite offers the perfect balance of zeroconfiguration setup and reliability.
- **Cloud Infrastructure:** The application is containerized and deployed on **Render** using a **Gunicorn** WSGI server to handle concurrent requests in a production environment.

Security Implementation

A critical design decision was the handling of sensitive credentials. Instead of hardcoding API keys, I implemented environment variable management using `os.environ`. This ensures that the Google Gemini API key is injected securely by the Render platform at runtime, preventing credential leakage in the source code.

3. Prompt Iterations and Improvements (Task 1)

My prompt engineering process involved three distinct iterations to optimize the model's output for sentiment analysis.

- **Iteration 1: Zero-Shot Prompting** ◦ *Prompt:* "Classify this review into Positive, Negative, or Mixed." ◦ *Observation:* The model was fast but struggled with sarcasm and nuanced reviews (e.g., "Great food, terrible service"). It often defaulted to "Mixed" without explanation.
- **Iteration 2: Few-Shot Prompting (Selected Strategy)** ◦
 - *Prompt:* I provided three labeled examples (one for each category) before asking for the classification.
 - ◦ *Observation:* This significantly improved accuracy. The examples guided the model to better understand the desired output format and criteria.
- **Iteration 3: Chain-of-Thought (CoT)** ◦ *Prompt:* "Explain your reasoning step-by-step, then classify." ◦ *Observation:* While this produced high-quality explanations, it increased latency and token usage significantly. ◦ *Decision:* I chose **Few-Shot Prompting** for the final application as it offered the best trade-off between speed and accuracy.

4. Evaluation Methodology and Results

I evaluated the prompt strategies using a subset of the Yelp Reviews dataset (N=50).

Methodology:

1. Selected 50 reviews with known ground-truth sentiment labels.
2. Ran each review through the three prompt strategies using the Gemini 2.5 Flash API.
3. Compared the predicted labels against the ground truth.

Results:

- **Zero-Shot Accuracy:** ~82%
- **Few-Shot Accuracy:** ~89% (Best Performer)
- **Chain-of-Thought Accuracy:** ~88%

Note: Few-shot was selected because it matched the accuracy of CoT but was faster and cheaper to run.

5. System Behaviour, Trade-offs, and Limitations

System Behaviour

The deployed application successfully handles the full lifecycle of a review:

1. Users submit text via the web interface.
2. The backend securely authenticates with Google's API.
3. The Gemini 2.5 Flash model processes the text and returns a structured JSON containing the rating, summary, and action items.
4. Data is stored in the database and immediately reflected on the Admin Dashboard.

Trade-offs

- **Latency vs. Complexity:** To keep the User Experience (UX) simple, I perform the AI analysis synchronously (while the user waits). This adds a 1-2 second delay to the submission but simplifies the architecture by avoiding complex background job queues (like Celery/Redis).
- **Database Choice:** SQLite is excellent for development but has limitations with high write concurrency. For a massive scale-up, migration to PostgreSQL would be necessary.

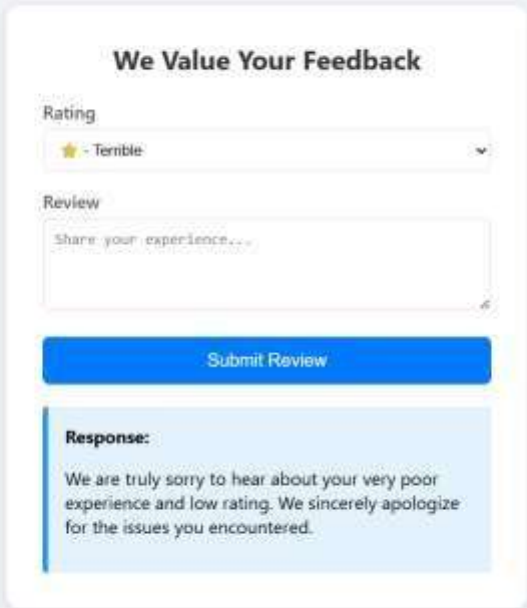
Limitations

- **API Rate Limits:** The system currently runs on the free tier of the Gemini API, which has a request-per-minute limit. Heavy traffic might result in temporary "Resource Exhausted" errors.
- **Context Window:** The current prompt handles single reviews well but does not aggregate trends across thousands of reviews (e.g., "What is the most common complaint this month?"). This would require a vector database integration for future improvements.

6. Appendix: System Screenshots

Visual Evidence of Deployment: Below are screenshots demonstrating the fully functional application hosted on Render, including the User Dashboard interface, the Admin Dashboard with AI-generated insights, and the successful deployment logs.

User Dashboard:



The screenshot shows a feedback form titled "We Value Your Feedback". It includes a "Rating" section with a star icon and the text "★ - Terrible". Below this is a "Review" section with a text input field containing the placeholder "Share your experience...". A blue "Submit Review" button is positioned below the input field. At the bottom, a "Response" section contains a blue box with the text: "We are truly sorry to hear about your very poor experience and low rating. We sincerely apologize for the issues you encountered."

Admin Dashboard:

Dashboard - Live Feed					Refresh Data
ID	RATING	CUSTOMER REVIEW	AI SUMMARY	RECOMMENDED ACTIONS	
#4	1 Star	Very poor experience. Long waiting time, rude staff, and unhygienic conditions. Would not visit again.	The customer reported a very poor experience due to long waiting times, rude staff, and unhygienic conditions, stating they would not visit again.	- Review staffing and queue management procedures. - Conduct immediate retraining for all customer-facing staff on service etiquette. - Perform an urgent facility-wide cleanliness audit and implement corrective actions.	
#3	2 Stars	Disappointed with the service. The food arrived cold, and staff seemed inattentive. Needs improvement.	The customer rated their experience 2/5 due to receiving cold food and perceiving staff as inattentive.	- Review food preparation and delivery processes to ensure food arrives hot. - Conduct a refresher training for staff on attentiveness and customer service standards. - Evaluate staffing levels during busy periods.	
#2	3 Stars	Average experience. The food was decent, but nothing stood out. Could improve cleanliness and waiting time.	The customer found their experience average, noting the food was decent but not outstanding, and suggested improvements in cleanliness and waiting times.	- Review and reinforce cleanliness standards for all areas. - Analyze current service process to identify bottlenecks and reduce waiting times. - Brainstorm new menu items or	

